

Gold or Objectives? A Study on League of Legends Win Probability

Abstract

In League of Legends, the most commonly cited measure of who is winning a game is gold difference. This project examines whether this is true, or whether it is just a convenient approximation, and seeks to determine if there is a better proxy for win probability. Two win probability models, a logistic regression baseline and an XGBoost gradient boosted tree ensemble, are trained on a per minute snapshot of 295,852 snapshots drawn from 9,596 real ranked matches on the 16.17 patch. Both models are then subjected to feature ablations that remove gold, experience, and level features. Both models reach a test AUC around 0.816-0.818, and removing either feature group costs a statistically significant but small amount of predictive power (0.0058-0.0082 AUC), with gold-related features carrying somewhat more standalone signal (94-99% of full-model AUC alone) than objective-related features (94% alone). Objective control's independent value is roughly seven to eighteen times larger in close games than in blowouts, and it disappears almost entirely at the highest rank tiers, where gold and objectives are tightly coupled. The results indicate that gold difference is a strong but incomplete proxy for win probability, understating a team's true chances specifically when the game is still closely contested.

Background

League of Legends is a multiplayer online battle arena (MOBA) game in which two teams of five players, the blue side and red side, compete on a map to destroy the opposing team's nexus. Each player controls a champion and earns gold and experience over the course of a match by defeating minions, monsters, and enemy champions. Gold is spent on items that make a champion stronger, and experience raises a champion's level, unlocking and strengthening their abilities. Beyond fighting for gold and experience directly, teams can also fight over neutral map objectives, epic monsters such as dragons and Rift Herald, and defensive structures such as towers, both of which grant a team a material advantage when secured, as well as less tangible map control and tempo. A match ends when one team destroys the other's nexus.

In League of Legends, the most commonly cited measure of who is winning a game is gold difference. It is shown in the overlays on broadcasts across nearly every level of play, from professional matches to

the replay interface in the game client itself, presented as a readout of which team currently holds the advantage. The reasoning behind it is straightforward, as gold purchases grant items, which in turn increase a champion's power, and greater power tends to win fights, so the team with more gold is presumed to be the team that is winning. Under this framing, every other event in a game, dragons, towers, Rift Herald, first blood, matters only to the extent that it helps a team accumulate or protect a gold advantage. If this framing is correct, a win probability model requires little more than the gold difference between the two teams to perform well.

This project treats that assumption as a hypothesis to be tested rather than a fact to be assumed, since it is rarely examined directly in casual analysis of the game.

Problem Statement

The core question is:

Does win probability in League of Legends reduce to gold, or does objective control carry significant predictive value independent of the gold it generates?

The question is difficult to answer because gold, experience, kills, and objectives are all heavily correlated with each other in a real match. For example, a team that secures a dragon has usually just won a fight, and a team that wins a fight usually gains gold and experience in the same instant. Furthermore, teams that have accumulated more gold tend to also have more objectives.

Data Collection

Match data is obtained from the Riot Games API, drawn from Ranked Solo/Duo games. Matches are stratified across ten skill brackets, Iron through Diamond, evenly split across their four divisions, along with Master, Grandmaster, and Challenger as single apex brackets, with a target of 1,000 matches per bracket.

League of Legends regularly receives biweekly patches, and thus changes considerably from patch to patch. Champion balance shifts, item costs and effects may be changed, as well as structural elements such as tower plating or objective bounties. Collecting data across multiple patches over an extended window would confound rank with patch, since different brackets would inevitably complete collection at different times. To avoid this, data collection is restricted to a single patch (16.17).

For each collected match, the Riot API's match timeline is used to reconstruct the state of the game at one minute resolution, including cumulative gold, experience, level, creep score, damage dealt and taken, kills, deaths, assists, and objective and tower state (dragons, heralds, barons, elders, plates, and

each of the eleven individual towers), tracked separately for both teams and broken down by role where available. Each row in the resulting dataset represents a snapshot of both teams at one minute of one match, labeled with the eventual winner. Feature engineering then derives the difference and per minute rate versions of these raw counts.

Modeling Methodology

Both models are evaluated on an identical held out test set. The split is done at the match level, meaning that a full match and all of its per minute rows are assigned entirely to either the training or test partition.

Logistic regression fits a linear decision boundary over standardized features with an L2 penalty. It cannot represent interactions between features or nonlinear effects, but its coefficients are directly interpretable, since a feature's sign and magnitude indicate precisely how much it changes the predicted log odds of a blue side win.

XGBoost is a gradient boosted ensemble of decision trees, capable of representing nonlinear effects and interactions between features. It can capture patterns such as a dragon advantage mattering more when the game is otherwise close, a relationship a linear model cannot.

Both models are trained on the same features and evaluated against the same held out test set as described above. Each is tuned using an extensive grid search, as shown in Figure 1.

Model	Hyperparameters Searched	Best Configuration
Logistic Regression	inverse regularization strength (C) = 0.001 L2 penalty	C = 0.001, L2 penalty
XGBoost	maximum tree depth (3, 4, 6, 8) learning rate (0.01, 0.03, 0.05, 0.1) row subsample ratio (0.6, 0.7, 0.85, 1.0) column subsample ratio per tree (0.7, 0.85, 1.0)	maximum tree depth = 8 learning rate = 0.05 row subsample ratio = 0.6 column subsample ratio = 0.7

Figure 1. Hyperparameter grids searched for each model, selecting on validation log loss.

Both models are evaluated using the same criteria, using overall AUC on the test set, as well as AUC separated by game minute bucket, allowing a model's performance to be examined separately for the early, middle, and late stages of a game.

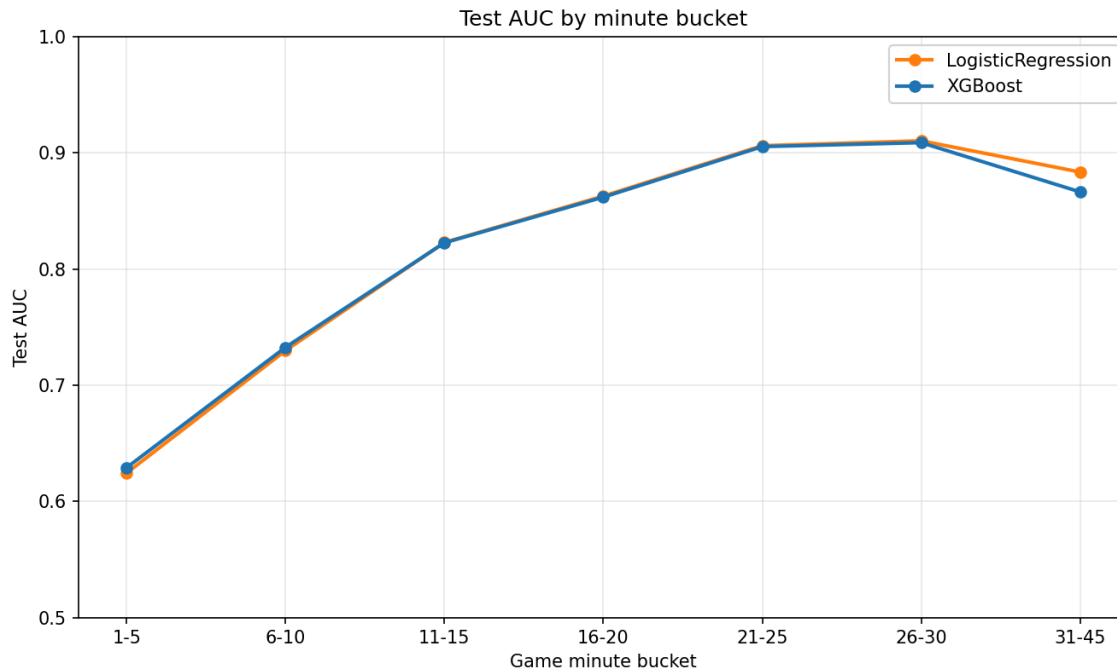


Figure 2. Test AUC by game-minute bucket

Logistic Regression Baseline

On the held-out test set, the logistic regression baseline achieved an AUC of 0.8181. Its calibration curve tracks the diagonal closely across all ten probability bins. Performance rises steadily with game time, from an AUC of 0.6244 in minutes 1-5 to a peak of 0.9104 in minutes 26-30, before tapering slightly to 0.8835 in the 31-45 minute bucket. This is consistent with early-game snapshots carrying little decided state, and with long games being comparatively rarer and more back-and-forth.

The largest predictors by coefficient magnitude were the difference in dragons taken (+0.4016) and the difference in team gold (+0.3532), followed by the difference in bot lane gold (+0.2400), the difference in nexus towers destroyed (-0.2057), and the difference in jungler level (+0.1888). Notably, an epic monster feature outweighs the raw gold difference at the top of the ranking.

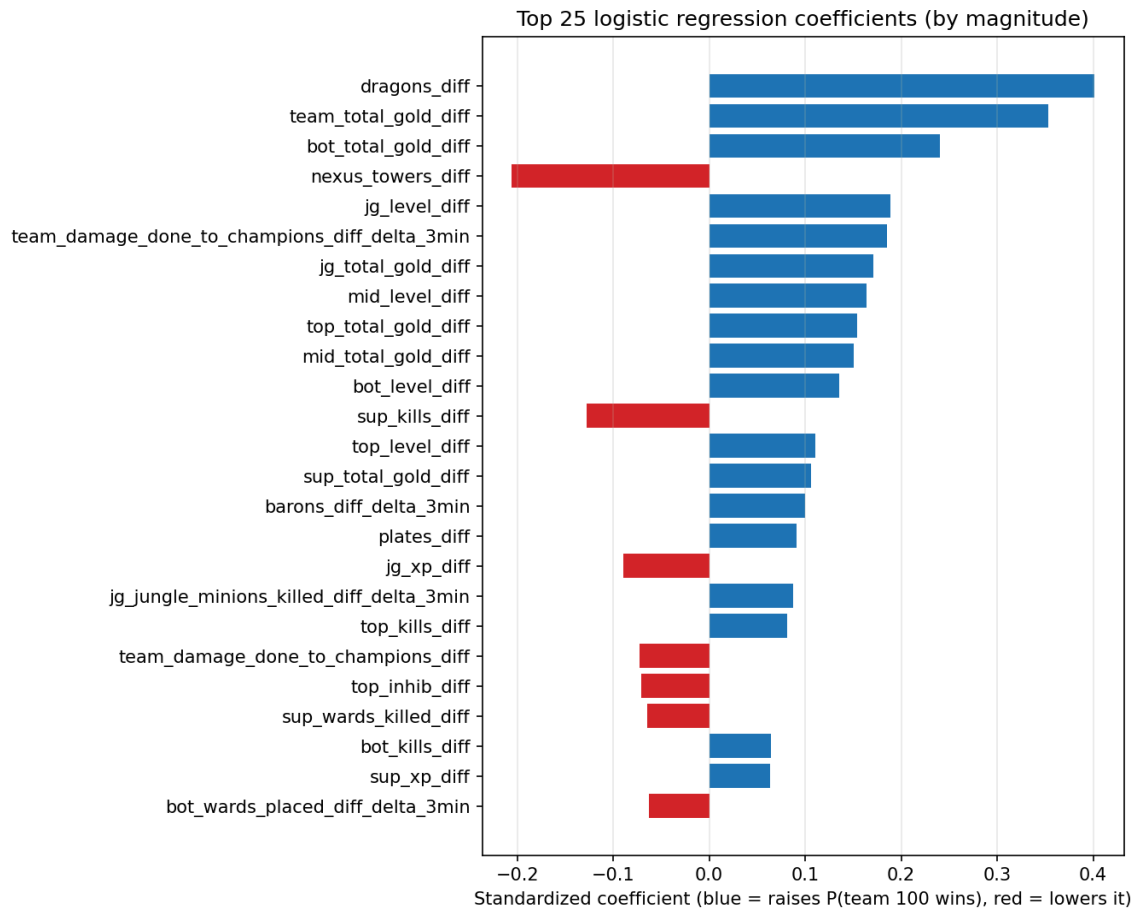


Figure 3. The twenty five largest logistic regression coefficients by magnitude

XGBoost

On the held-out test set, XGBoost achieved an AUC of 0.8160, behind logistic regression's 0.8181. Performance by minute bucket followed the same shape as logistic regression, rising from an AUC of 0.6290 in minutes 1-5 to a peak of 0.9088 in minutes 26-30, before dropping back down to 0.8664 in the 31-45 minute bucket. The dip in the final bucket is more pronounced for XGBoost than for logistic regression (0.8664 versus 0.8835), consistent with a tree ensemble generalizing worse into that bucket's smaller sample of games.

By gain, the top features were the difference in team gold, the difference in team experience, the difference in team kills, the difference in team deaths, and the difference in dragons taken. This overlaps with the logistic regression ranking, which shares team gold and dragon control at the top but otherwise weights lane-level gold and jungler level rather than team experience, kills, or deaths. As both

architectures independently surface team gold and dragons as leading predictors, the ablation findings that follow are unlikely to be an artifact of either model's biases.

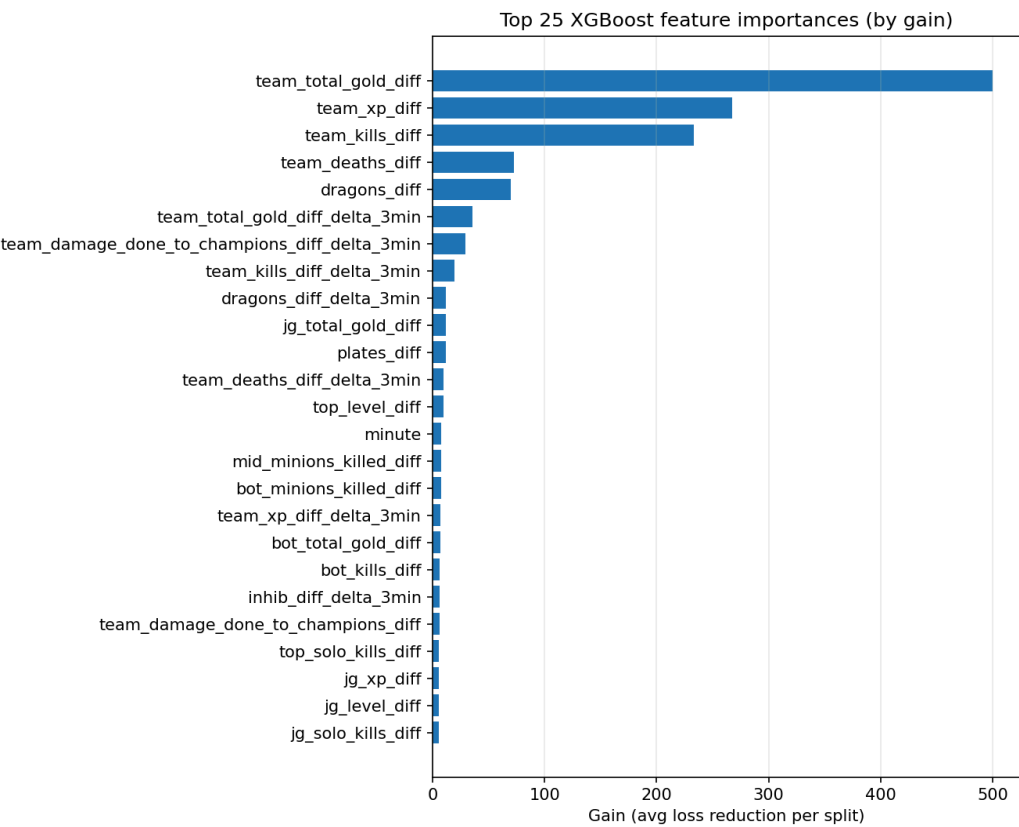


Figure 4. The twenty five most important XGBoost features ranked by gain.

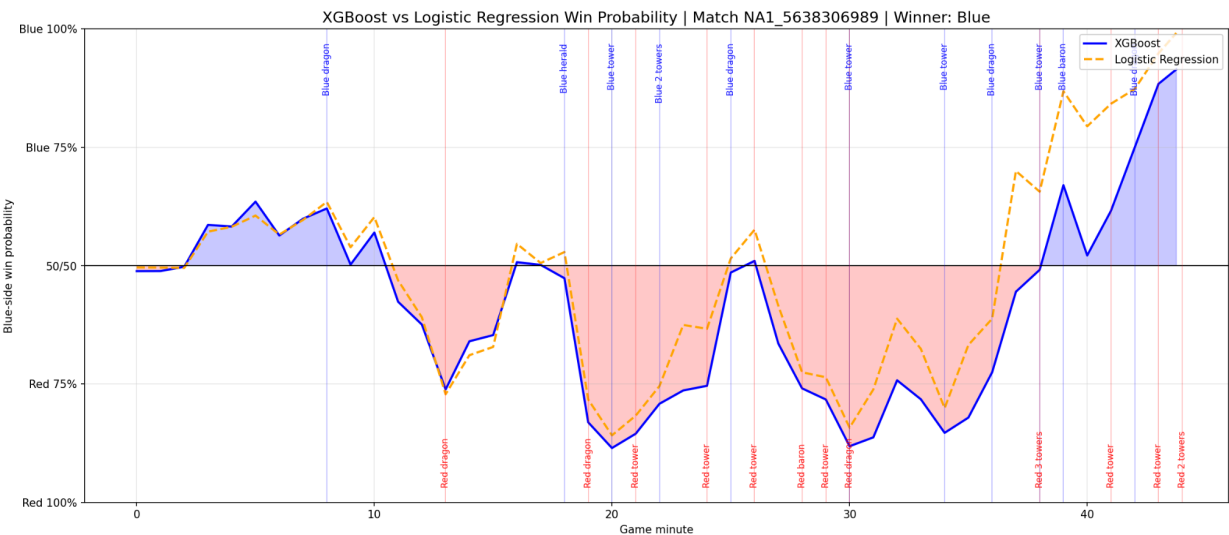


Figure 5. Blue side win probability over the course of a single match as predicted by both models

Results and Analysis

Gold alone does not fully account for win probability. Removing the economy features costs 0.0060 AUC for logistic regression and 0.0082 for XGBoost, while removing objectives costs a comparable 0.0058 and 0.0065. Both drops are statistically significant, so neither group clearly dominates the other, as shown in Figure 6.

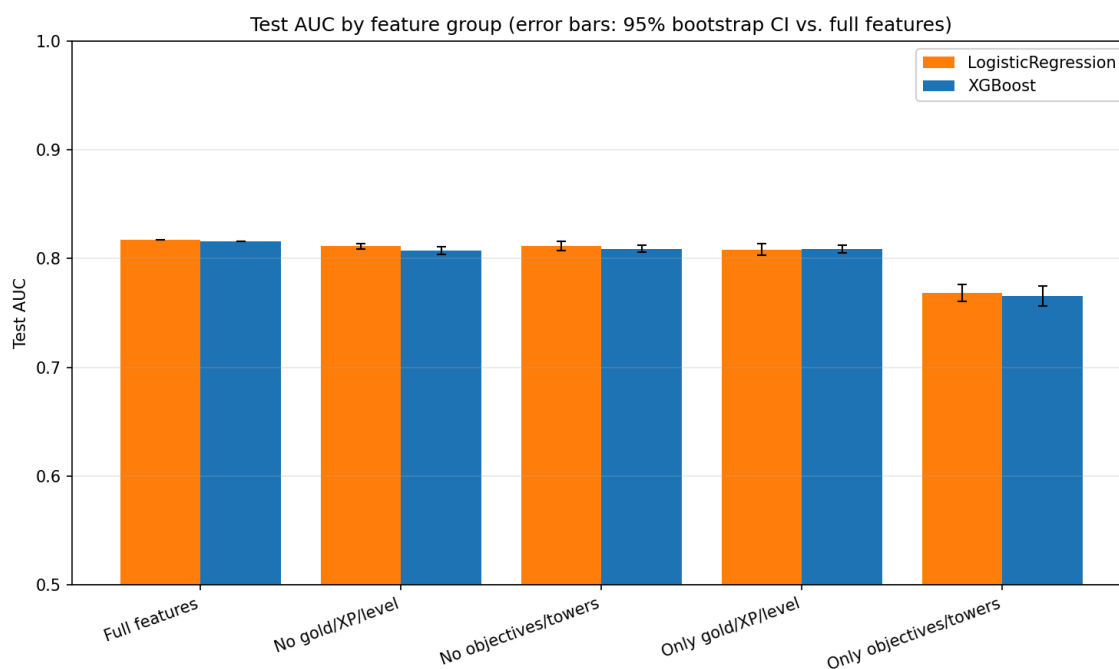


Figure 6. Test AUC by feature group, with 95% bootstrap confidence intervals against the full-feature model.

However, the two groups are not independent. Training on economy features alone reaches an AUC of 0.8083 for logistic regression and 0.8087 for XGBoost, recovering 98.9 and 99.2 percent of the full model's AUC. Training on objectives alone reaches only 0.7685 and 0.7657, recovering 94.0 and 93.9 percent. Economy features alone come much closer to full performance than objectives alone do, so while removing either group costs the model a similar amount, gold-related features carry more standalone predictive power than objective-related features do.

Game Closeness

Breaking the results down by game closeness, now measured against fixed gold-lead thresholds (close under 2,500 gold, medium between 2,500 and 7,500, blowout above 7,500) adds further nuance. In close games, the objectives-ablation gap runs from 0.0102 to 0.0125 AUC, and narrows further to 0.0007 to 0.0015 in blowouts. Objective information matters most while the outcome is still undecided and least once one team has built a decisive lead. For the objectives-only model, the pattern moves in the same direction rather than the opposite one: the AUC gap relative to the full model is largest in close games at 0.0698 to 0.0702, falls to 0.0362 to 0.0372 in medium games, and falls further still to 0.0042 to 0.0051 in blowouts. Objectives alone are least sufficient on their own in the closest games, not the most sufficient.

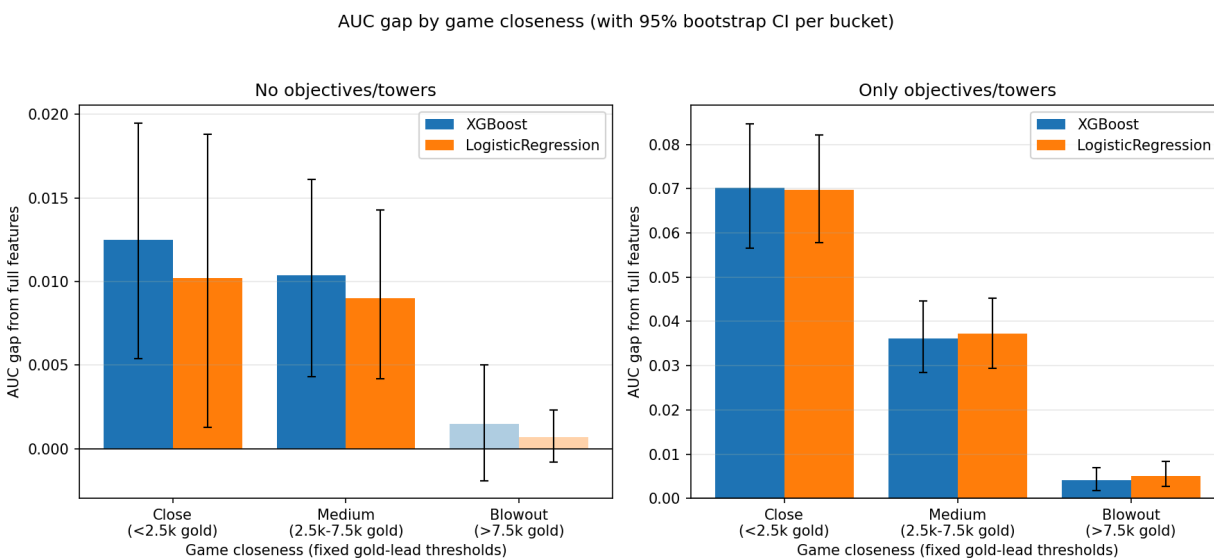


Figure 7. AUC gap from the full-feature model by game closeness, with 95% bootstrap confidence intervals.

For the objectives ablation, the AUC gap is largest in close games (median absolute gold difference of about 732, gap of 0.0125 for XGBoost and 0.0102 for logistic regression) and drops close to zero in blowouts (median absolute gold difference of about 9,550, gap of 0.0015 and 0.0007). Objective control matters most exactly when there is no economic lead yet to fall back on. Isolating objectives alone tells a consistent story this time rather than a flip side: standalone objective-only AUC retains the least of the full model's performance in close games (89.5% for XGBoost and 89.6% for logistic regression), more in medium-closeness games (96.0% and 95.9%), and nearly all of it in blowouts (99.6% and 99.5%). Once a game has already been decided, objective control alone tracks that outcome closely, while in a genuinely close game neither gold nor objectives in isolation tell the full story.

Rank

Splitting the same two ablations out by rank tier, from Iron through Challenger, adds a third axis to this picture. The cost of removing objectives is fairly steady through most of the middle of the ladder, running roughly 0.0024 to 0.0129 AUC across Iron through Master for both models, with the exception of Silver, where the logistic regression gap briefly turns negative (-0.0010). The effect flattens out for good only at the very top. In Grandmaster it is close to zero for both models (a gap of -0.0014), and in Challenger it stays near zero for XGBoost (0.0016) and turns slightly negative for logistic regression (-0.0057). At the highest ranks, removing objectives from the model costs essentially nothing. This may be due to the extremely small player base that consists of extremely strong players, who are more able to leverage comparatively smaller gold leads.

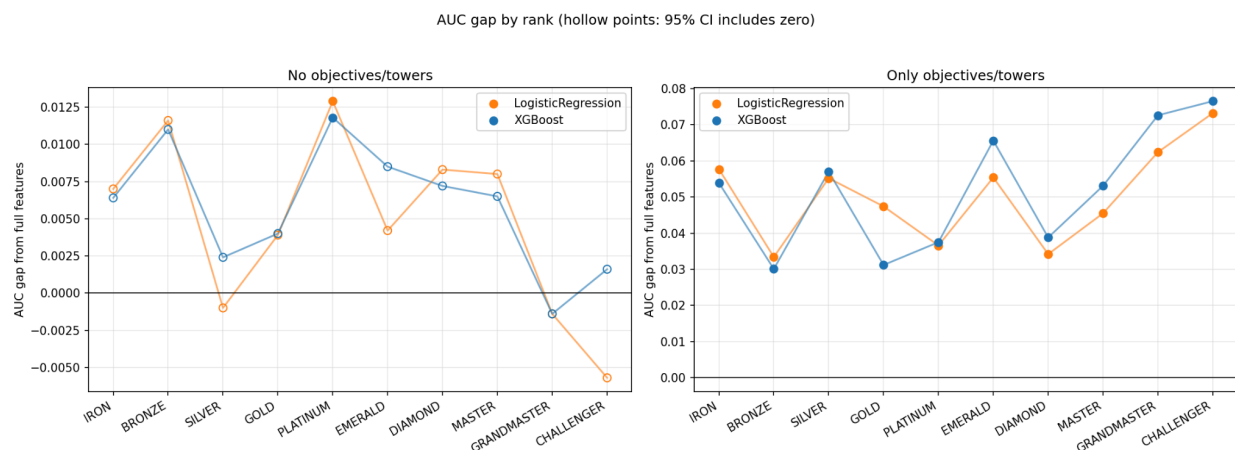


Figure 8. AUC gap from the full-feature model by rank tier. Hollow points indicate a 95% confidence interval that includes zero.

The objectives-only model tells a matching story from the other direction. Its retained AUC sits around 92 to 96 percent through most of the ladder, but drops to its lowest points at the top, 91.3 percent for XGBoost and 92.5 percent for logistic regression in Grandmaster, and 90.3 and 90.7 percent in Challenger. Objectives alone explain the least at the skill levels where they matter least for the full model too. Read together, these two results point the same way. At the top of the ladder, objective control and gold have become so tightly coupled that gold already encodes almost everything objectives would add, so the two ablations essentially agree with each other for the first time across the whole rank range.

Statistical significance in this report should be read carefully alongside effect size. With 295,852 snapshots drawn from 9,596 matches, even small, practically modest differences in AUC are easily

detected by the bootstrap confidence intervals: a gap of 0.0058–0.0082 AUC being "significant" means it is reliably nonzero, not that it is large. In absolute terms, no single feature group moves the model's discrimination by more than about one percentage point of AUC in the full-data comparisons. The more meaningful signal in this report is therefore not whether an effect is statistically distinguishable from zero, but how its magnitude varies across game closeness and rank.

Limitations

This ablation study measures association after controlling for other features present in the model, not a causal effect from a randomized experiment. A real match cannot be replayed with its objectives removed, so "how much AUC survives without objectives" describes how much independent predictive signal the removed features carried in this dataset, not what would happen to a team's actual win probability if a dragon were somehow taken away from them.

The feature groups used for ablation are defined by a keyword match against feature names, gold, xp, and level for economy, and dragon, herald, baron, elder, plate, and tower for objectives. This is simple and fully transparent, but it also means any feature whose name does not contain one of these keywords, or whose name coincidentally does, is grouped by name rather than by a deeper semantic check.

The dataset is restricted to a single patch, which keeps rank from being confounded with patch but also means the results describe this patch, not every point in the game's balance history. Whether the same asymmetry between gold and objectives holds on other patches, where champion balance, item costs, or objective bounties have been tuned differently, is outside the scope of this project.

Conclusion

Gold difference is a strong but incomplete proxy for win probability. Gold, experience, and level features alone reach 0.808–0.809 AUC, recovering 99 percent of the full model, which is why it dominates as a broadcast heuristic. But objective and tower features carry real, independent signals: removing them costs a small but statistically significant 0.0058–0.0065 AUC, and an objectives-only model reaches 0.766–0.769 AUC on its own, far above the 0.51 baseline. That signal is not evenly distributed, as it is concentrated in close games (roughly seven to eighteen times larger than in blowouts) and largely vanishes at the top of the ladder, where gold and objectives have become so tightly coupled that gold already encodes nearly everything an objective would add.

So win probability does not reduce to gold. Gold is the stronger, cheaper signal, but it understates a team's true chances exactly when the game is still contested and below the highest ranks.

References

- [1] Riot Games. (n.d.). Riot Games API. Retrieved from <https://developer.riotgames.com/>
- [2] Chen, T., and Guestrin, C. (2016). XGBoost: A scalable tree boosting system. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 785-794. <https://doi.org/10.1145/2939672.2939785>
- [3] Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825-2830. <https://www.jmlr.org/papers/v12/pedregosa11a.html>
- [4] Streamlit Inc. (n.d.). Streamlit documentation. Retrieved from <https://docs.streamlit.io/>